

Reviewer Pack — Minimal Rank of Non-Archimedean Valuations over DPLL Search ...

Entry 9336a97eb5e6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 12:40:20 UTC

Minimal Rank of Non-Archimedean Valuations over DPLL Search Trees

Entry ID: 9336a97eb5e6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 12:40:20 UTC

1. Conjecture

Field A (mathematical branch): Non-Archimedean Analysis **Field B** (complexity object): Complexity Theory: DPLL Search Trees

Statement:

[‘For a given satisfiability problem in CNF, the minimal rank of its associated non-archimedean valuation is monotonically related to the depth of the DPLL search tree for that problem.’, ‘Formally, if $V \models D \models \text{CNF} \models \text{DPLL} \models \text{min_rank}(V) = \Theta(\text{depth}(D))$.’, ‘Moreover, there exists a constructive mapping from a CNF instance to a non-archimedean valuation such that the rank of this valuation is uniquely determined by the structure of the DPLL tree.’]

Rationale (proposer’s reasoning):

[‘Non-archimedean analysis provides a framework for understanding the local properties of functions and their valuations. By applying this to DPLL search trees, we may uncover new insights into the complexity of solving satisfiability problems.’, ‘The connection between non-archimedean valuations and DPLL tree depth suggests that the valuation could serve as an invariant capturing the structural complexity of the problem, potentially leading to a proof of $P \neq NP$.’]

Taxonomy category: NonArchimedeanAnalysis (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): fd7c755b160f6da8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all 30 random CNF instances, the ratio of $\text{min_rank}(V)$ to $\text{depth}(D)$ falls within $[0.5, 1.5]$. It is falsified if this ratio exceeds 1.5 for any seed or falls below 0.5 for any seed.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "non-archimedean valuation" AND "DPLL search tree" AND CNF" - "minimal rank" AND non-Archimedean AND complexity theory" - CNF satisfiability problem AND depth of DPLL AND non-archimedean valuation

Top relevant hits considered: - [http://arxiv.org/abs/1104.4279v2] Satisfiability of Acyclic and Almost Acyclic CNF Formulas - [http://arxiv.org/abs/0807.1282v2] Satisfiability of Almost Disjoint CNF Formulas - [http://arxiv.org/abs/1801.03588v1] Deterministic search for CNF satisfying assignments in almost polynomial time

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
import sys

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        cnf = []
        for _ in range(m):
            clause = [random.randint(1, n), -random.randint(1, n)]
            cnf.append(clause)
        return cnf

    def dpll(cnf, assignment):
        if not cnf:
            return True
        unit_clause = next((c for c in cnf if len(c) == 1), None)
        if unit_clause:
            literal = unit_clause[0]
            if literal < 0:
                literal = -literal
            new_assignment = assignment.copy()
            new_assignment[literal] = True
```

```

        if dpll([c for c in cnf if literal not in c and -literal not in c], new_assignment):
            return True
        else:
            new_assignment[literal] = False
            if dpll([c for c in cnf if literal not in c and -literal not in c], new_assignment):
                return True
            else:
                return False
    pure_literal = next((l for l in range(1, n+1) if all(l in c or -l in c for c in cnf)), None)
    if pure_literal:
        new_assignment = assignment.copy()
        new_assignment[pure_literal] = True
        if dpll([c for c in cnf if pure_literal not in c and -pure_literal not in c], new_assignment):
            return True
        else:
            new_assignment[pure_literal] = False
            if dpll([c for c in cnf if pure_literal not in c and -pure_literal not in c], new_assignment):
                return True
            else:
                return False
    literal, _ = random.choice(cnf)
    new_assignment = assignment.copy()
    new_assignment[literal] = True
    if dpll([c for c in cnf if literal not in c and -literal not in c], new_assignment):
        return True
    else:
        new_assignment[literal] = False
        if dpll([c for c in cnf if literal not in c and -literal not in c], new_assignment):
            return True
        else:
            return False

def non_archimedean_valuation(cnf, assignment):
    valuation = 0
    for clause in cnf:
        if any(assignment.get(lit, False) for lit in clause):
            valuation += 1
    return valuation

n = random.randint(5, 40)
m = random.randint(n, n * 3)
cnf = generate_cnf(n, m)
assignment = {i: False for i in range(1, n+1)}

depth = 0
stack = [(cnf, assignment)]
while stack:
    cnf, assignment = stack.pop()
    if not cnf:
        break
    unit_clause = next((c for c in cnf if len(c) == 1), None)
    if unit_clause:
        literal = unit_clause[0]
        if literal < 0:
            literal = -literal
        new_assignment = assignment.copy()

```

```

        new_assignment[literal] = True
        stack.append((cnf, new_assignment))
        break
    pure_literal = next((l for l in range(1, n+1) if all(l in c or -l in c for c in cnf)), None)
    if pure_literal:
        new_assignment = assignment.copy()
        new_assignment[pure_literal] = True
        stack.append((cnf, new_assignment))
        break
    literal, _ = random.choice(cnf)
    new_assignment = assignment.copy()
    new_assignment[literal] = True
    stack.append((cnf, new_assignment))
    break

min_rank = non_archimedean_valuation(cnf, assignment)

return {
    "metric_name": "min_rank_over_depth",
    "metric_value": Fraction(min_rank, depth),
    "instances_tested": 1,
    "conjecture_holds": 0.5 <= Fraction(min_rank, depth) <= 1.5,
    "counterexample": "" if 0.5 <= Fraction(min_rank, depth) <= 1.5 else f"min_rank={min_rank}"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        seeds = [2*i + 1 for i in range(5, 6)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

```

Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a38f7b0b.py", line 126, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a38f7b0b.py", line 113, in run_trial
    "metric_value": Fraction(min_rank, depth),
                     ^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(0, 0)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which prevents us from verifying if the ratio of min_rank(V) to depth(D) falls within the specified range for all seeds. | next: Investigate the cause of the crash and rerun the test with proper error handling to ensure it can complete without interruption.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11879
2	preregistration	ollama_remote	glm4:latest	0	0	5809
3	novelty	ollama_remote	glm4:latest	0	0	4723
4	novelty	ollama_remote	glm4:latest	0	0	5746
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17642
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13711
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11783
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14565
9	judge	ollama_remote	glm4:latest	0	0	10822

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 96679 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/9336a97eb5e6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/9336a97eb5e6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9336a97eb5e6.tar.gz` (if generated)